# MarkingForge

Studio deployment guide - for administrators

3DS MAX 2027 · VERSION 1.5.71

Installing MarkingForge on many workstations without dialogs, giving a whole team one dial layout and the standard shortcuts - or leaving the keys to you - handing out dial files and colour presets, pointing 3ds Max at a prepared dial folder, updating and removing it - with the script `Studio\Deploy-MarkingForge.ps1` that comes in the download.

# Contents

1. What gets installed, and where
2. Silent installation with the script
3. Deployment tools
4. One dial layout for the whole team
5. Dial files, every-dial folders and colour presets
6. A prepared dial folder - MARKINGFORGE_CONFIG_DIR
7. Updating and removing
8. Verifying a workstation
9. Troubleshooting for administrators
10. Support, bug reports and feature requests

# What gets installed, and where

| Part | Location | Who |
|---|---|---|
| The plugin (one folder) | `%ProgramData%\Autodesk\ApplicationPlugins\MarkingForge` | All users of the machine - needs administrator rights |
| | `%APPDATA%\Autodesk\ApplicationPlugins\MarkingForge` | One Windows user - no rights needed |
| Each user's dials and settings | `%LOCALAPPDATA%\Autodesk\3dsMax\2027 - 64bit\ENU\en-US\plugcfg\MarkingForge` | Per user, always. The English 3ds Max folder shown; other languages use their own `ENU\xx-XX`. The dials can be moved to another folder with `MARKINGFORGE_CONFIG_DIR` (chapter 6). |

The plugin folder also holds the **dial library** (`dials\Built-in`, `Extra`, `Packs` - the same files as `Dials` in the download) that the editor's **Dial library...** offers. It is replaced with every update; put your own dial files elsewhere.

### What is in a user's settings folder

| File | What it is |
|---|---|
| `menus.json` | The dials: what each one shows, its other sets (the pages under the mouse wheel), mouse buttons, the colours of every dial and of each dial and page. |
| `usage.json` | How often each direction was used (the editor's Usage column). Local - not worth distributing. |
| `editor_colors.json` | The menu editor's own colours (**Editor colours...**). |
| `dial_color_presets.json`, `editor_color_presets.json` | The user's own colour presets for the dials and for the editor. |
| `hotkeys\MarkingForge_Hotkeys.hsx` | The shortcut file the editor writes when it applies shortcuts. 3ds Max then uses it as the user's **active** hotkey set - all of their shortcuts, not only MarkingForge's. Always in this folder, even with `MARKINGFORGE_CONFIG_DIR` set (1.4.7). |
| `menu_log.txt`, `shortcuts_first_start.done`, `*_shortcuts.flag` | The start-up log, the record that the first-start shortcuts ran, and the request the script leaves for the next start (`-SetupShortcuts` / `-NoShortcuts`). |

3ds Max loads the plugin from Application Plugins through its `PackageContents.xml`. Nothing is written into the 3ds Max program folder or the registry, and MarkingForge makes no network connections - no licence server, no telemetry.

> **Never install in both scopes.** 3ds Max would load two copies and report a duplicate plugin at start-up. The script refuses to do it (exit code 4).

> Licensing: one licence per person who uses MarkingForge (see LICENSE.txt). The deployment method does not change that.

# Silent installation with the script

Run from the unzipped download, in an elevated PowerShell for all-users installs. 3ds Max must be closed.

```
powershell -ExecutionPolicy Bypass -File "Studio\Deploy-MarkingForge.ps1" -Scope AllUsers
```

## Parameters

| Parameter | Type | What it does |
|---|---|---|
| `-Scope` | string | AllUsers or CurrentUser (default AllUsers). |
| `-Source` | string | The MarkingForge package folder (the one holding PackageContents.xml). Default: "Manual install\MarkingForge" of the download this script came in. |
| `-StudioMenus` | string | A studio dial layout for the CURRENT user: a menus.json or a preset saved with the editor's "Save preset...". An existing menus.json is backed up first. |
| `-SetupShortcuts` | switch | Give the current user the standard shortcuts (Alt / Ctrl+Alt / Shift+Alt + 1..8) on their next 3ds Max start - free keys only. Since 1.1.0 a first start does this by itself when no dial has a key yet; the switch also fills free slots of a user who already has some. |
| `-NoShortcuts` | switch | Leave the current user's keys alone on the next start - the first start would otherwise give the standard keys to a user without any. |
| `-Uninstall` | switch | Remove MarkingForge from this scope. Users' own settings are kept. |
| `-SettingsOnly` | switch | Only the per-user part (-StudioMenus / -SetupShortcuts): the plugin is neither installed nor checked. For log-on scripts on machines where the plugin was installed for all users. |
| `-KeepExistingMenus` | switch | With -StudioMenus: give the studio layout only to users who have NO menus.json yet - so a log-on script never overwrites what a user built. |
| `-AllowScripts` | switch | With -StudioMenus: accept a layout whose items run MAXScript (script or value items). Without it such a layout is refused, as the editor refuses a preset's scripts unless the user consents. |
| `-DryRun` | switch | Show what would be done; change nothing. |
| `-Force` | switch | Install even when a copy exists in the other scope (not recommended: 3ds Max would load the plugin twice). |

## Exit codes

| Code | Meaning |
|---|---|
| `0` | Success. |
| `1` | Stopped by an error the output names: contradicting switches (-SetupShortcuts with -NoShortcuts), a file -Uninstall could not remove, or a studio layout that could not be copied. Fix what the message says and run again. |
| `2` | No MarkingForge package at the source folder. |
| `3` | 3ds Max is running - it holds the plugin file open. Close it and run again. |
| `4` | MarkingForge is installed in the other scope - 3ds Max would load it twice. Uninstall that copy first. |
| `5` | The update did not complete (a copy error or a file that does not match the package by SHA-256 - a disk, network or permission problem). The previous version is restored automatically; the output says so, or where its backup is. |
| `6` | The studio layout file is missing, not valid JSON or not a MarkingForge layout (schema 1). Checked before anything is installed, so nothing was changed. |

| | |
|---|---|
| `7` | The studio layout has items that run MAXScript (script or value items). Checked before anything is installed. Check them, then run again with -AllowScripts to install it anyway. |
| `8` | Files of the installed version are open in another program - a PDF of the documentation in a viewer, say - or cannot be changed. The output names them. Checked before anything is changed, so nothing was changed: close them and run again. |

## What the script does, in order

1. Stops if 3ds Max is running (it holds the plugin file open) - not with `-SettingsOnly`, which does not touch the plugin.
2. With `-StudioMenus`: checks the studio layout (exit codes 6 and 7) - before anything is installed, so a bad layout leaves the workstation unchanged.
3. Checks the package: `PackageContents.xml` and `plugins\MarkingForge.gup`; reads its version.
4. Stops if MarkingForge is installed in the other scope.
5. Stops if a file of the installed version is open in another program, a PDF viewer say (exit code 8) - before anything is changed.
6. Backs up the installed version to `%TEMP%` (never next to the plugin, where 3ds Max would load it as a second copy).
7. Removes the previous version in this scope and copies the new one.
8. Verifies every file by SHA-256 against the package. If the copy or the check fails, the previous version is put back and the script ends with code 5 - a failed update never leaves a workstation without a working plugin.
9. Optionally prepares the current user's dial layout (`-StudioMenus`) and what the next start does with the shortcuts (`-SetupShortcuts` or `-NoShortcuts`).

Add `-DryRun` to any command to see these steps without changing anything.

> **Shortcuts on the first start.** Since 1.1.0 the first 3ds Max start after installation gives every dial its standard key (Alt / Ctrl+Alt / Shift+Alt + 1-8) by itself - once per user and 3ds Max version, only when no dial has a key yet, and skipping keys used by something else. A studio with its own key scheme adds `-NoShortcuts`; `-SetupShortcuts` fills the free dials also for a user who already has some keys. Use one or the other: the script refuses the pair with an error.

# Deployment tools

Any tool that can run a PowerShell script with administrator rights works - Microsoft Intune, Configuration Manager, PDQ Deploy, a Group Policy start-up script. Copy the download folder to a share and run:

```
powershell -NoProfile -ExecutionPolicy Bypass -File
"\\server\apps\MarkingForge_1.5.71\Studio\Deploy-MarkingForge.ps1" -Scope AllUsers
```

| Tool setting | Value |
|---|---|
| Install command | the line above |
| Uninstall command | the same with `-Uninstall` |
| Detection | file `%ProgramData%\Autodesk\ApplicationPlugins\MarkingForge\PackageContents.xml` containing `AppVersion="1.5.71"` |
| Success exit code | 0 |
| Requirement | 3ds Max 2027 installed; 3ds Max not running (exit code 3 otherwise - retry later) |

# One dial layout for the whole team

1. On one workstation, build the layout in **MarkingForge > Menu editor...** and **Save**.
2. Copy that user's `menus.json` (or a file saved with **Save preset...** - see the table below for the difference) to a share, for example `\\server\3d\MarkingForge\studio_menus.json`.
3. Give it to users from a **log-on script** (it runs as the user, so it reaches the user's own settings folder):

```
powershell -NoProfile -ExecutionPolicy Bypass -File
"\\server\apps\MarkingForge_1.5.71\Studio\Deploy-MarkingForge.ps1" ^
    -SettingsOnly -StudioMenus "\\server\3d\MarkingForge\studio_menus.json" -KeepExistingMenus -
SetupShortcuts
```

| Switch | Effect |
|---|---|
| `-SettingsOnly` | Only the user part - the plugin is not installed or touched. |
| `-StudioMenus` | Installs the layout as the user's menus.json; an existing one is backed up first. |
| `-KeepExistingMenus` | Only for users who have no layout yet - a log-on script never overwrites what a user built. |
| `-AllowScripts` | Accept a studio layout with items that run MAXScript (script or value items). Without it such a layout is refused (exit code 7), as the editor refuses a preset's scripts unless you consent. |
| `-SetupShortcuts` | On the next start the user's free dials get Alt / Ctrl+Alt / Shift+Alt + 1-8. Keys the user already uses are skipped. Without it the first start does the same for a user who has no MarkingForge key at all. |
| `-NoShortcuts` | Leave the user's keys alone on the next start - for a studio that assigns its own keys. |

> To push a **new** studio layout to everybody, run once without `-KeepExistingMenus`. Each user's previous file stays next to it as `menus.json.bak_before_studio_<date>_<id>`.

| Studio layout file | Carries | Does not carry |
|---|---|---|
| `menus.json` of the workstation | Every dial, its sets (pages) in their order, mouse buttons, the colours of every dial and of each dial and page. | Keyboard shortcuts (they live in 3ds Max's hotkey set - use `-SetupShortcuts`), usage counters. |
| A **Save preset...** file | What each dial shows, its sets (pages), mouse buttons, the colours of every dial and of each dial and page (1.5.28). | Keyboard shortcuts, usage counters, the editor's own colours. |

Individual users can still change their dials in the editor; a preset loaded with **Load preset...** replaces only the dials it contains.

# Dial files, every-dial folders and colour presets

Pieces a user loads in the editor themselves - for a team that keeps its own layouts and takes only what it wants from the studio. Put them on a share; nothing here is installed by the script.

| To hand out | Made with (editor) | The user loads it with |
|---|---|---|
| One dial | **Save dial...** (under **Sets of this dial**), or right-click the dial > **Save Menu N (...) to a file...** - a `.mfdial` file | **Load dial...** or right-click a dial > **Load a dial file into Menu N...** - in place of what it shows, or as a new set (a page under the mouse wheel) |
| Every dial | **Save every dial...** - a folder `MarkingForge dials <date>` with one `NN name.mfdial` per dial and per set, and the list `MarkingForge dials.json` | **Load every dial...** - what each dial showed, its sets in their order, the dials that showed the default menu and the colours of every dial. Dials the folder does not hold stay as they are. |
| A colour look for the dials or the editor | **Colours...** or **Editor colours...** > **Presets** > **Export this preset to a file...** - a `.mfcolors` file | **Presets** > **Import a preset from a file...**, then **Use this** - a dial look can go on every dial, one dial or the pages of a dial |

- **Only some dials from a folder:** delete the other `.mfdial` files **and** the list `MarkingForge dials.json`. Without the list the folder is read from the file names (the first file of a dial is the one it shows) and the colours of every dial are not changed. With the list in place a file it names that is missing stops the whole load.
- **Scripts:** MAXScript items in dial files from outside are removed unless the user chooses to keep them - prefer 3ds Max commands in anything you hand out.
- **Loaded dials are not written until Save** - the user can look first and press **Reload from file** to throw them away.
- **Colour presets for everybody:** instead of importing one by one, a set of presets can be copied as `dial_color_presets.json` (or `editor_color_presets.json`) into the user's settings folder. That replaces the user's own presets of that kind; the built-in looks (30 for the dials, 22 for the editor) are always there.

# A prepared dial folder - MARKINGFORGE_CONFIG_DIR

When the environment variable `MARKINGFORGE_CONFIG_DIR` names a folder that exists, 3ds Max reads and writes the dials there instead of in the user's `plugcfg\MarkingForge` - `menus.json` and everything beside it: `usage.json`, the editor's colours and the colour presets. Use it to start a workstation on a prepared set of dials, or to keep a test 3ds Max away from a user's own dials.

**Shortcuts never move (1.4.7).** The shortcut file, `hotkeys\MarkingForge_Hotkeys.hsx`, always stays in the user's own `plugcfg\MarkingForge`, whatever the variable says. 3ds Max remembers its active hotkey set once per 3ds Max version, for every session of it - so a hotkey file in the prepared folder would be used by the user's own 3ds Max too, and removing that folder would leave 3ds Max with no shortcuts at all. Up to 1.4.6 the file followed the variable.

```
setx MARKINGFORGE_CONFIG_DIR "D:\MarkingForge\Dials"        (this user)
setx /M MARKINGFORGE_CONFIG_DIR "D:\MarkingForge\Dials"     (every user of the machine - administrator)
```

| Rule | Detail |
|---|---|
| Read when 3ds Max starts | Set the variable, then start 3ds Max again (a variable set by Group Policy needs a new log-on). |
| An absolute path to an existing folder | A relative path or a folder that does not exist is IGNORED - the usual folder is used and nothing is created. Quotes around the value are taken off. |
| Said, not silent | **MarkingForge > Diagnostics > Configuration status** has a `config dir` line: which folder is used, or why the variable was ignored. |
| An empty folder | Gets the built-in dials on the first start. To start on a prepared set, copy a `menus.json` into the folder first. |
| The user needs write access | Every Save in the editor and the usage counters are written there. |
| What stays in plugcfg\MarkingForge | The shortcut file `hotkeys\MarkingForge_Hotkeys.hsx` and its backups (1.4.7), `menu_log.txt`, the first-start record and the `-SetupShortcuts` / `-NoShortcuts` flags - those switches work as usual. |

> **One folder per user.** `-StudioMenus` writes to `plugcfg\MarkingForge`, which 3ds Max does not read while the variable is in use - put the studio `menus.json` into the prepared folder instead. Do not point several users at one shared folder: they would save into one `menus.json`, and 3ds Max reloads `menus.json` whenever it changes, so one user's Save would change everybody's dials. Keep the master copy elsewhere and copy it into each user's folder.

# Updating and removing

| Task | Command | Notes |
|---|---|---|
| Update | run the install command from the new version's folder | Users keep their dials and shortcuts. |
| Remove | `-Uninstall -Scope AllUsers` | User settings stay in each profile (and in a `MARKINGFORGE_CONFIG_DIR` folder). They can be deleted **except** the `hotkeys` subfolder while 3ds Max uses its shortcut file as the user's active hotkey set - assign another set in Customize > Hotkey Editor first. |
| Move from per-user to all-users | `-Uninstall -Scope CurrentUser` (as that user), then install `-Scope AllUsers` | Avoids the double-load. |

# Verifying a workstation

In 3ds Max's MAXScript Listener (F11):

```
MarkingForge.version()        -- the installed version; "Unknown property" = not loaded
MarkingForge.configSummary()  -- the user's menus.json, its dials, any errors, the config dir
line
MarkingForge.configPath()     -- the menus.json in use (MARKINGFORGE_CONFIG_DIR or plugcfg)
MF_TOOLS_DIR                  -- where the editor is loaded from (the Application Plugins folder)
```

Without the Listener: **MarkingForge > Diagnostics > Configuration status** prints the same summary (and the experimental features). The start-up log `plugcfg\MarkingForge\menu_log.txt` records the menu being built and what the first start did with the shortcuts.

# Troubleshooting for administrators

| Symptom | Cause and fix |
|---|---|
| Exit code 3 | 3ds Max is running on the workstation. Schedule for when it is closed, or retry. |
| Exit code 4 | An older per-user installation exists. Remove it as that user (-Uninstall -Scope CurrentUser). |
| Duplicate plugin message at 3ds Max start | A copy of MarkingForge.gup in the 3ds Max plugins folder or in both scopes. Keep one. |
| Plugin not loaded | 3ds Max 2027 missing or the Visual C++ 2015-2022 x64 runtime missing - install it. |
| Studio layout not applied | The user already had menus.json and -KeepExistingMenus was set, 3ds Max runs in another language (settings folder ENU\xx-XX), or MARKINGFORGE_CONFIG_DIR points 3ds Max at another folder. |
| The prepared folder is not used | Configuration status says why: a relative path or a folder that does not exist (as 3ds Max sees it - a drive letter mapped later, a typo). 3ds Max reads the variable only when it starts. |
| Shortcuts not set up | Keys already used by something else are skipped on purpose; the first start sets keys only when no dial has one, and never after -NoShortcuts. The user can run MarkingForge > Set up standard shortcuts... - menu_log.txt says what the first start did. |
| -SetupShortcuts and -NoShortcuts together | The script stops with an error - they contradict each other. Pick one. |

> **Still stuck?** Open an issue at github.com/Forge-Plugins/MarkingForge/issues or write to forgeplugins@gmail.com. **MarkingForge > Report a bug...** puts your 3ds Max and MarkingForge details on the clipboard, ready to paste.

# Support, bug reports and feature requests

| | |
|---|---|
| ISSUES | github.com/Forge-Plugins/MarkingForge/issues |
| ROADMAP | trello.com/b/7gJVqG8Y/markingforge-features-roadmap |
| E-MAIL | forgeplugins@gmail.com |
| WEB | https://forgeplugins.tech (forgeplugins.tech - more plugins by the author) |

**Bug reports and feature requests** are best opened as issues on GitHub - the forms there ask for exactly what is needed, and you can follow the fix. Questions, and anything you would rather not post publicly (licence, purchase), go to the e-mail address. Every message is read.

## Reporting a bug

1. In 3ds Max choose **MarkingForge > Report a bug...** - it copies your 3ds Max version, MarkingForge version, configuration summary and start-up log to the clipboard. It works even when the plugin did not load.
2. Open github.com/Forge-Plugins/MarkingForge/issues and pick **Bug report** - or write an e-mail to forgeplugins@gmail.com instead. Both are links in the window that opens (since 1.5.20), one click each.
3. Say what you did, what you expected and what happened instead, then paste the details with **Ctrl+V**.
4. Add a screenshot if you can. When the problem is in one dial, attach a preset of it (editor > **Save preset...**).

## Suggesting a feature

Open an issue with the **Feature request** form, choose **MarkingForge > Suggest a feature...**, or simply write. Describe what you would like to do and how you do it today - a concrete example from your work says more than a list of wishes. Ideas for new built-in dials, commands and options are all welcome. What is already planned and in progress is on the **Roadmap** above - **MarkingForge > Roadmap (Trello)** opens it too.

Nothing is sent by the plugin itself: the details go to the clipboard and your own e-mail program, so you see everything that leaves your computer. MarkingForge has no network access.